Assignment 1
Reinforcement Learning

Tavishi Kaushik | October 26, 2026

Value and policy iteration; policy gradient without and with a baseline.

GitHub username: Tavishi-Kaushik

Repository name: rl-assignment-1

Repository: https://github.com/Tavishi-Kaushik/rl-assignment-1

Summary of findings

The shuttle MDP has a dispatch threshold of 13 passengers. Finite-horizon enumeration, value iteration and policy iteration agree within explicit numerical error bounds. CartPole improves over the random-action reference. A completed 4,000-episode DeepDish follow-up improves Pong scoring with a time baseline, but both new policies still lose all 500 final games. Correct implementation is not evidence that the task has been solved.

Frozen policy	Selected episode	Mean reward	Sample SD	Approx. 95% CI
CartPole: no baseline	600	141.990	43.098	[138.212, 145.768]
CartPole: time baseline	400	344.440	75.673	[337.807, 351.073]
Pong initial: no baseline	400	-20.406	0.774	[-20.474, -20.338]
Pong initial: time baseline	600	-20.552	0.629	[-20.607, -20.497]
Pong DeepDish: no baseline	4000	-20.286	0.866	[-20.362, -20.210]
Pong DeepDish: time baseline	4000	-18.926	1.687	[-19.074, -18.778]

Each row uses 500 held-out episodes with stochastic action sampling and no parameter updates. Sample SD uses N-1. The approximate interval is mean +/- 1.96 SD/sqrt(500). It measures evaluation uncertainty conditional on the selected model, not uncertainty across independently trained models.

Evidence and scope

CartPole uses the completed recovery runs, not reconstructed lost weights. The initial Pong experiments (seed 42, 1,000 episodes, batch 4) remain reported alongside the DeepDish follow-up (seed 7, 4,000 episodes, batch 8). These are different protocols, not a controlled single-factor comparison.

Included: all three shuttle plots, six learning curves with 100-episode averages, six 500-episode histograms, quantitative validation, edge cases, execution directions and a three-month validation plan.

1. Shuttle dispatch: model and enumeration

State s is the queue immediately before dispatch, s in {0,...,200}. Action a=0 waits; a=1 dispatches one shuttle of capacity K=15. One shuttle is available at every epoch; fleet size, travel time and vehicle location are not modeled. The arrivals A are independent over periods and uniform on {1,2,3,4,5}.

```
m(s,a) = max(s - 15a, 0)
c(s,a) = 100a + 2m(s,a)
s_next = min(m(s,a) + A, 200),  gamma = 0.95
```

Holding cost is charged after service and before the new arrivals. Overflow above 200 is dropped without a separate penalty. These are modeling assumptions, not inferred properties of a physical shuttle system. If a reward formulation is preferred, use r=-c and negate the reported cost values.

Part (a): T = 500

```
V_500(s) = 0
V_t(s) = min over a in {0,1}:
  c(s,a) + (0.95/5) sum_{A=1}^5 V_{t+1}(min(m(s,a)+A,200))
```

Backward induction enumerates both actions and all five arrivals at every state and time. This evaluates the finite-horizon problem exactly up to floating-point arithmetic; full action-sequence enumeration is unnecessary. There are 500 x 201 x 2 x 5 successor-value contributions. Exact action ties select waiting.

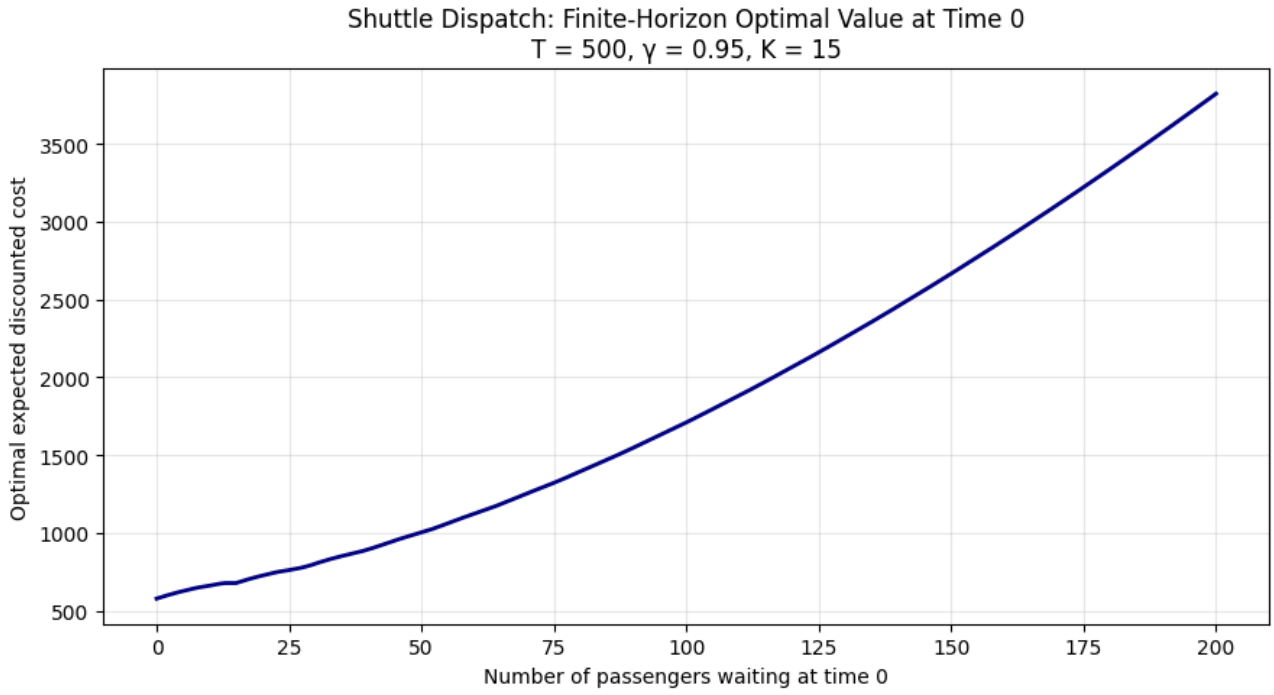


Figure 1. Optimal expected discounted cost at time 0 for the 500-period problem.

V_0(0)=576.670168 and V_0(20)=725.439658. The time-0 policy first dispatches at s=13 and dispatches at all larger states. The final two decision periods instead wait everywhere because the horizon has zero terminal cost.

2. Shuttle dispatch: value and policy iteration

Part (b): synchronous value iteration applies the Bellman minimization until the maximum update is below 1e-8. It converged in 429 iterations, with final update 9.922e-9. V*(0)=576.670168 and V*(20)=725.439658.

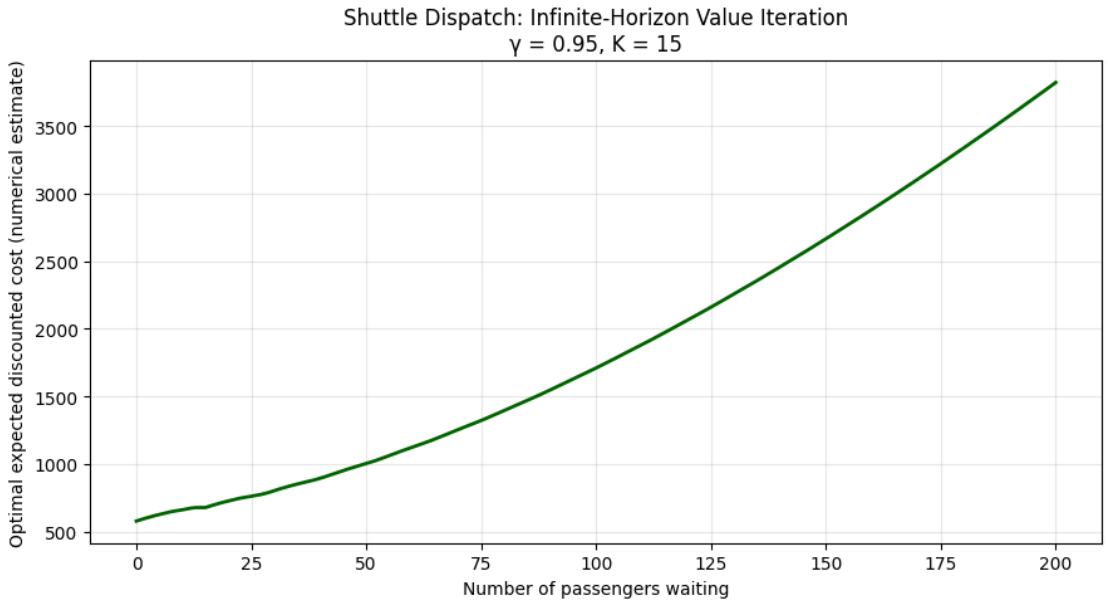


Figure 2. Infinite-horizon value-iteration cost function.

Part (c): policy iteration evaluates a fixed policy by solving (I-0.95 P_policy)V=c_policy, then improves actions greedily. Starting from waiting everywhere, changed-state counts were 196, 28, 22, 7, 1 and 0: convergence in six rounds. Ties within 1e-10 retain the existing action.

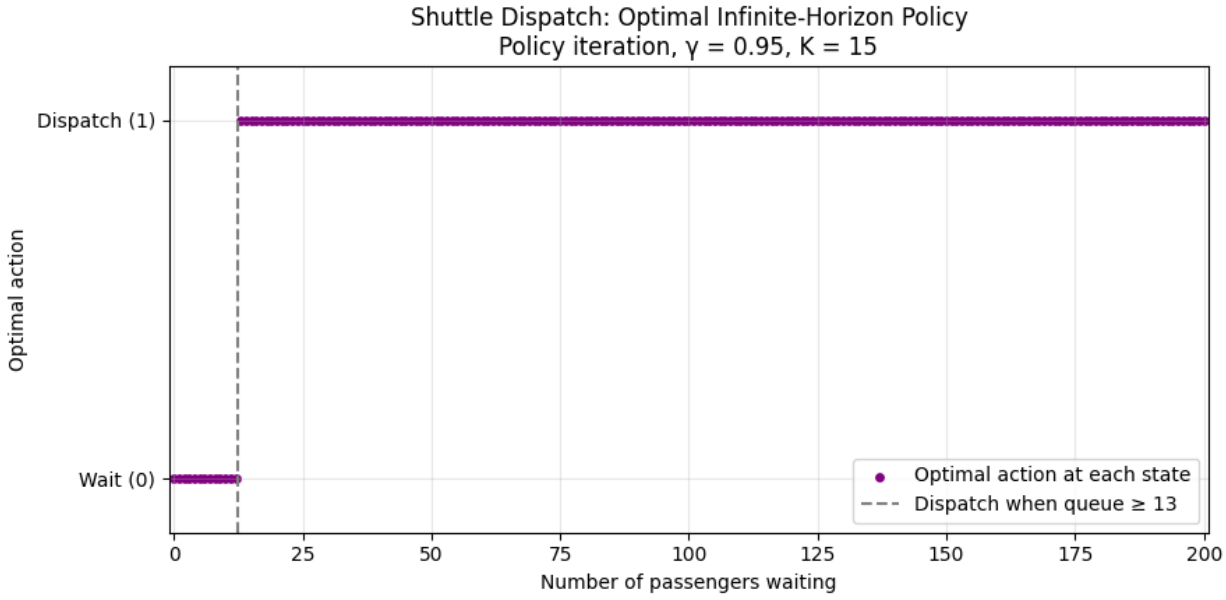


Figure 3. Policy-iteration optimum: wait for s<13 and dispatch for s>=13.

There are zero policy disagreements with value iteration, and the maximum value difference is 1.885e-7. A threshold below capacity is reasonable: discounted future waiting costs can justify dispatch before all 15 seats are occupied.

3. Shuttle edge cases and quantitative validation

Test and prediction	Observed evidence / why it validates the method
Empty / partial shuttle	m=max(s-15a,0) prevents negative queues. At s=20, costs are 40 (wait) and 110 (dispatch), with successors 21-25 and 6-10.
Queue cap and mass	At s=198 with no dispatch: [199,200,200,200,200]. State 200 receives probability 0.8. Repeated destinations are accumulated, not overwritten.
Final two periods	All states wait. Largest two-period holding saving is 30+0.95(30)=58.5<100. Last/penultimate value errors are both 0. At s=20 the two-period value is 83.70.
Free dispatch: c_f=0	All 200 nonempty states dispatch; V(0)=0 because arrivals never exceed capacity. This tests the dispatch-cost tradeoff.
Free holding: c_h=0	All 201 states wait; maximum |V|=0. Dispatching has no compensating benefit.
Zero discount factor	At gamma=0 only immediate cost matters; all states wait and V(s)=2s exactly. Maximum error is 0.
Zero capacity: K=0	Dispatch cannot reduce the queue, so all states wait. At s=200, V=400/(1-0.95)=8000; computed error 1.893e-7.
Independent enumeration	Enumerate 2 x 2^5=64 two-period contingent plans at six representative states. Their minimum agrees with the original backward-induction implementation.
Cap sensitivity	Changing the cap from 200 to 400 gives zero value differences and zero policy disagreements over states 0-50. This is local, not a claim about states near the cap.

Numerical bounds and a simulation check

The Bellman residual is 9.426e-9, giving ||V-V*||_infinity <= residual/(1-gamma) =1.885e-7. A conservative cost bound c<=500 gives a 500-period omitted tail 500(0.95^500)/(1-0.95)=7.274e-8. The observed finite/VI difference 1.836e-7 is below the combined bound 2.613e-7; disagreement in the last digits is expected.

A separate 5,000-trajectory rollout from an empty queue, using the optimal policy for 300 periods, gave mean 576.289616, SD 31.616584 and SE 0.447126. Its approximate 95% interval is [575.413249,577.165983]. The remaining-tail bound is 0.002075, and the analytic infinite-horizon cost is within the tail-adjusted interval. This checks the transition/cost implementation empirically, in addition to Bellman consistency.

4. Policy gradient: objective and implementation

For episode length L, the objective is the expected start-of-episode discounted reward. The return-to-go and minimized REINFORCE loss are:

```
G_t = sum_{k=t}^{L-1} gamma^(k-t) r_k
J(theta) = E[sum_{t=0}^{L-1} gamma^t r_t]
Loss = -(1/B) sum_{episodes i} sum_t
       gamma^t [G_(i,t) - b_t] log pi_theta(a_(i,t) | s_(i,t))
```

Part 1 sets b_t=0. Part 2 uses a lagged time-dependent baseline. The outer gamma^t is retained for the stated episodic objective. Returns are not centered or normalized; there is no entropy bonus, critic bootstrap or reset of returns at Pong point boundaries. Categorical logits define the two-action stochastic policy [4]. Gradients are averaged across episodes, not divided by each episode length.

Setting	CartPole-v1	ALE/Pong-v5
Observation / network	4 -> 128 Tanh -> 2; 898 trainable parameters	6,400 -> 128 ReLU -> 2; 819,586 parameters
Discount / optimizer	gamma=0.95; Adam, lr=0.01	gamma=0.99; Adam, lr=0.001
Completed budgets	1,000 episodes; batch 8; 125 updates per method	Initial: 1,000 / batch 4 / 250 updates. DeepDish: 4,000 / batch 8 / 500 updates.
Policy actions	0 and 1	Network outputs 0/1 map to environment 2/3 only
Evaluation	50 episodes per checkpoint; 500 final	30 episodes per checkpoint; 500 final

Gymnasium provides the required CartPole-v1 and ALE/Pong-v5 IDs [1,2]; the latter is Pong-v5 in the ALE namespace, not a switch to v4. Pong keeps frameskip=4 and repeat_action_probability=0.25. Initial experiments ran on CPU. The additional DeepDish study used exactly one GTX 1080: CPU rollouts and CUDA gradient computation. The objective, network, action restriction and environment settings were unchanged.

Pong image processing

Copy the RGB frame, crop rows 35:195, take every second row/column in channel 0, remove background values 144 and 109, and map other pixels to 1. The network receives the difference of consecutive 80x80 float32 images, flattened to 6,400 inputs. The first difference of every episode is zero. The difference representation conveys motion but can discard stationary objects; it is a modeling choice and a limitation.

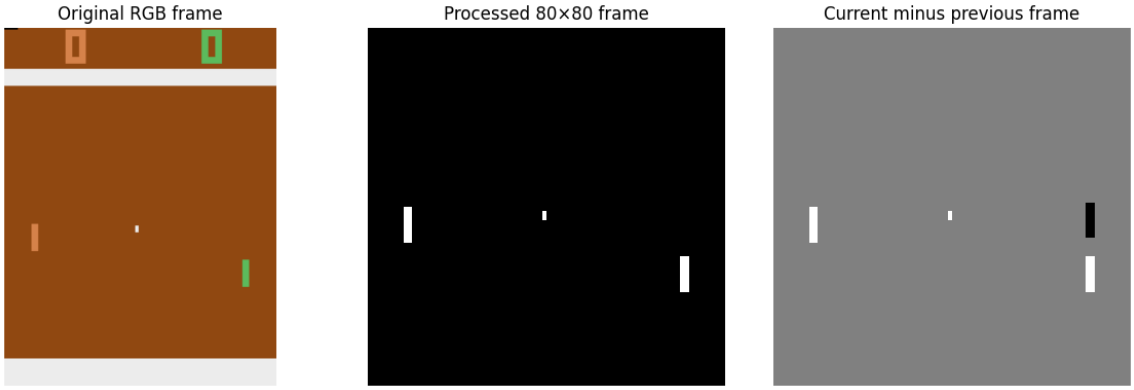


Figure 4. Saved preprocessing diagnostic: RGB frame, binary image and frame difference.

5. Part 1(a): CartPole without a baseline

The recovery experiment uses initialization seed 42 and 1,000 episodes at lr=0.01. Training, validation and final-evaluation seed ranges are disjoint. A historical lr=0.003 comparison did not improve validation performance, so the assignment-default 0.01 was retained. That comparison is a pilot, not an independent replication of this recovery run.

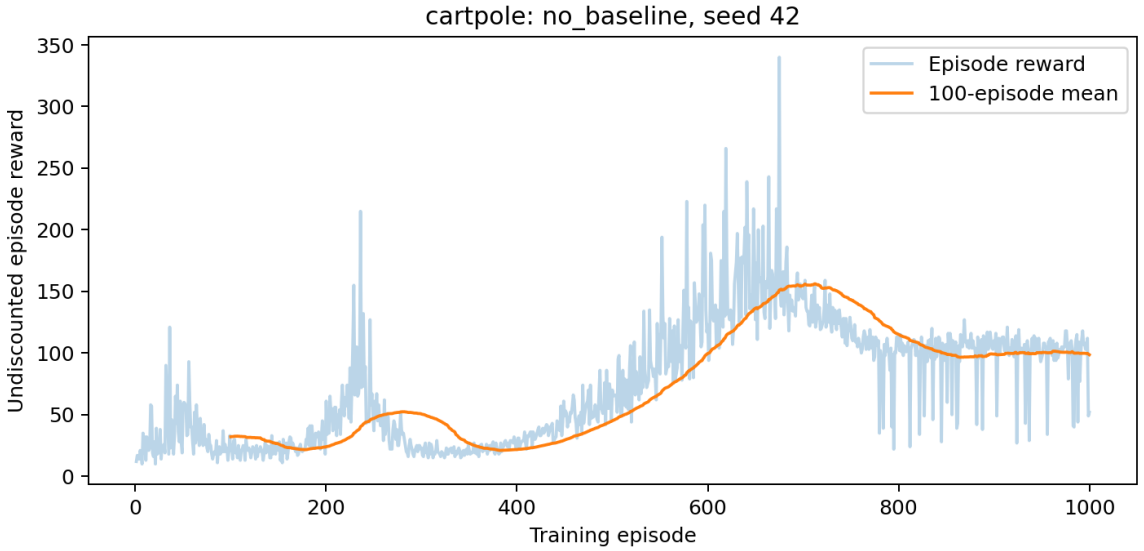


Figure 5. CartPole Part 1 training reward with the trailing 100-episode mean.

First-100 mean=32.10; last-100 mean=98.63; highest 100-episode mean=156.19, ending at episode 712. The curve is unstable rather than monotonically improving. Checkpoint 600 was selected by held-out validation mean, not by the final test or best single episode.

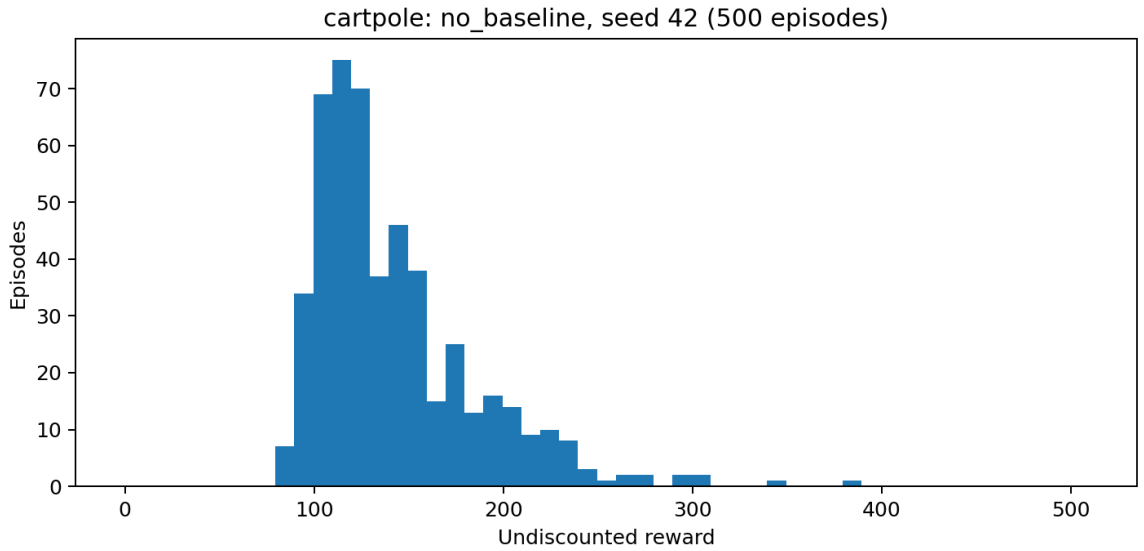


Figure 6. CartPole Part 1: 500 frozen-policy evaluation episode rewards.

Final mean=141.99; sample SD=43.10; range 82-383; 0/500 reach the 500-step limit. The historical 100-episode random-action reference had mean 20.44 and SD 9.93. This supports useful learning, but not reliable completion of CartPole; the random reference is not a newly matched 500-episode control.

6. Part 2(a): CartPole with a time baseline

The baseline is an exponential moving average of returns at each time index, initialized to zero. For each batch, use the old b_t for every actor gradient; after the Adam step, update b_t <- 0.9 b_t + 0.1 mean(G_t over episodes that reached t). Unvisited entries stay unchanged. The baseline is detached and does not use the current action; hence its score-function contribution has expectation zero. A baseline of zero exactly recovers the no-baseline loss and gradient.

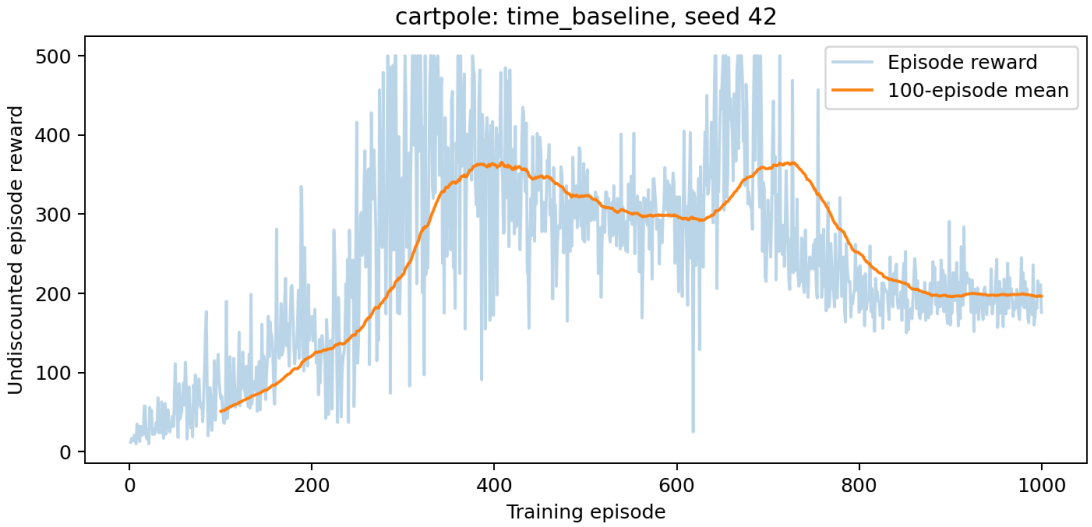


Figure 7. CartPole Part 2 training rewards with the trailing 100-episode mean.

Architecture, initialization seed, Adam settings, batch size and episode budget match Part 1. First-100 mean=51.04; last-100 mean=196.43; best 100-episode mean=365.22 (window ending 727). Checkpoint 400 had the highest validation mean, 344.66.

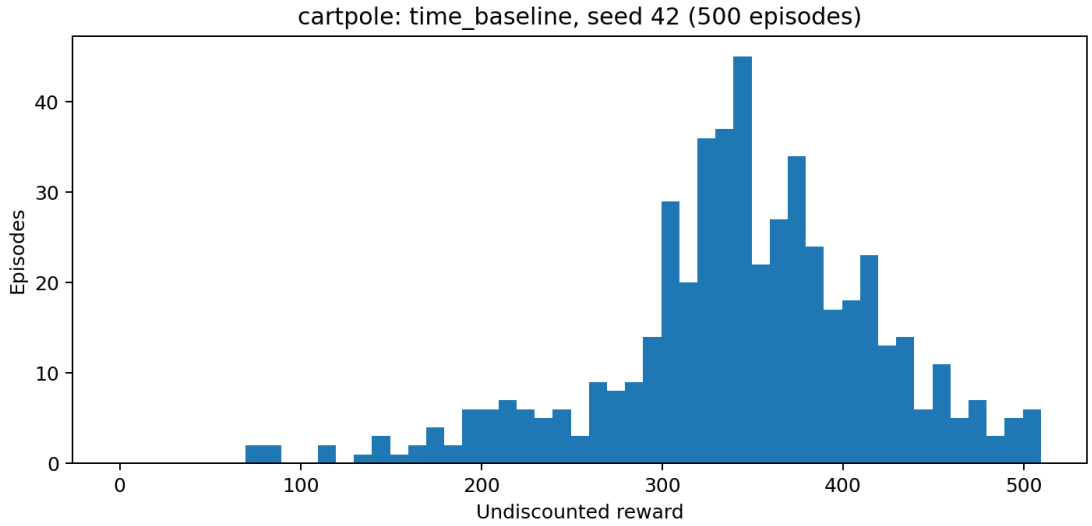


Figure 8. CartPole Part 2: 500 held-out rewards from selected checkpoint 400.

Final mean=344.44; sample SD=75.67; range 73-500; 6/500 (1.2%) reach the time limit. The paired mean advantage over Part 1 is 202.45, with approximate 95% CI [195.51,209.39], conditional on these selected policies. Reward SD is larger despite lower gradient-estimator variance; they measure different quantities.

7. Part 1(b): initial Pong without a baseline

Only actions 2 (RIGHT) and 3 (LEFT) are available to the policy. A real-environment check detected ball motion and scoring without FIRE. Ten random-action episodes gave mean -20.60, SD 0.52, mean length 972.1 and zero truncations. A 100-episode lr=0.01 pilot ended with last-20 mean -21.00. A fresh lr=0.001 run was therefore used for the 1,000-episode experiment; it remained weak.

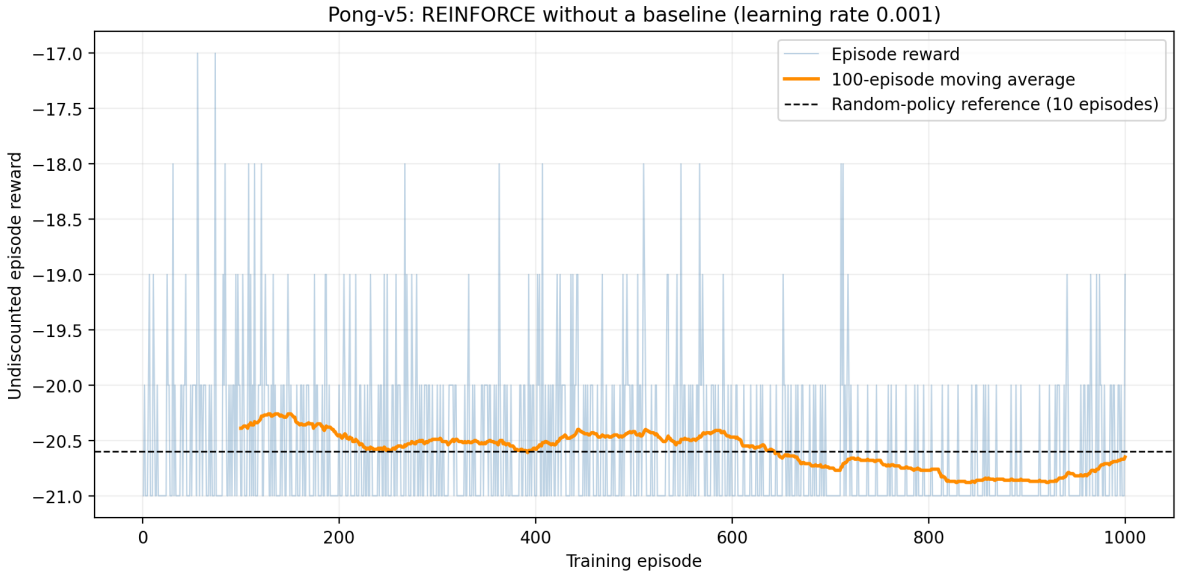


Figure 9. Initial seed-42 Pong Part 1 training score and 100-episode moving average.

First-100 mean=-20.39; last-100 mean=-20.65; best 100-episode mean=-20.26 (window ending 128). Starting discounted return changed from -1.3536 to -1.2145, so the optimized objective and full-game score need not improve together.

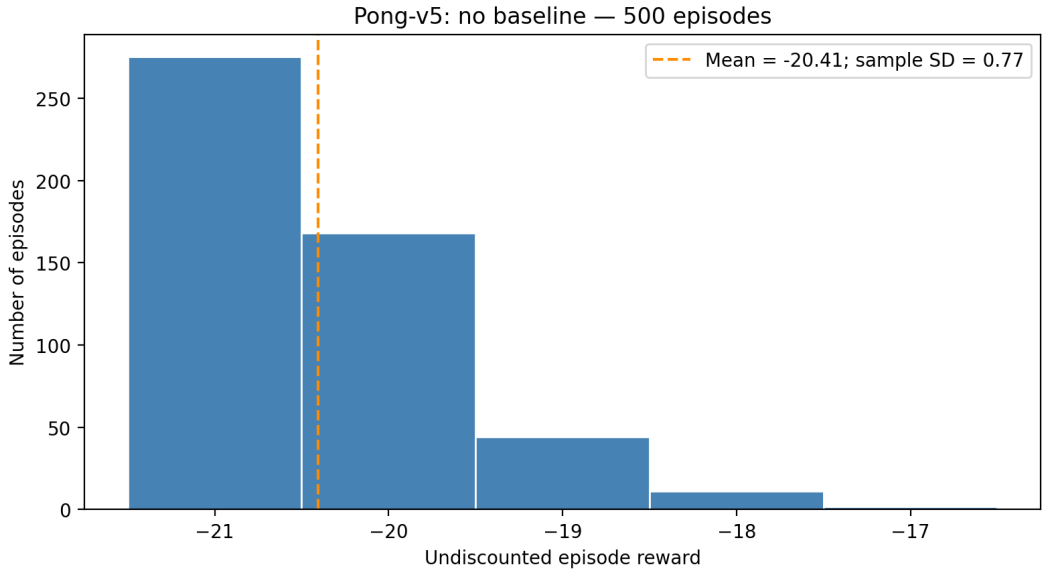


Figure 10. Pong Part 1: 500 held-out scores at checkpoint 400.

Validation selected checkpoint 400: its mean score tied checkpoint 200, with a better discounted-return tie-break. Final mean=-20.406; sample SD=0.774; range -21 to -17. All 500 games were losses and none was truncated. These results do not establish effective Pong play or a convincing improvement over the small random reference.

8. Part 2(b): initial Pong with a time baseline

The same lagged EMA baseline (alpha=0.1), loss convention and post-actor update timing are used for Pong. The baseline vector grows to cover new episode lengths, with zero initial values for new entries. Only visited time indices are updated. All other main hyperparameters match the no-baseline experiment.

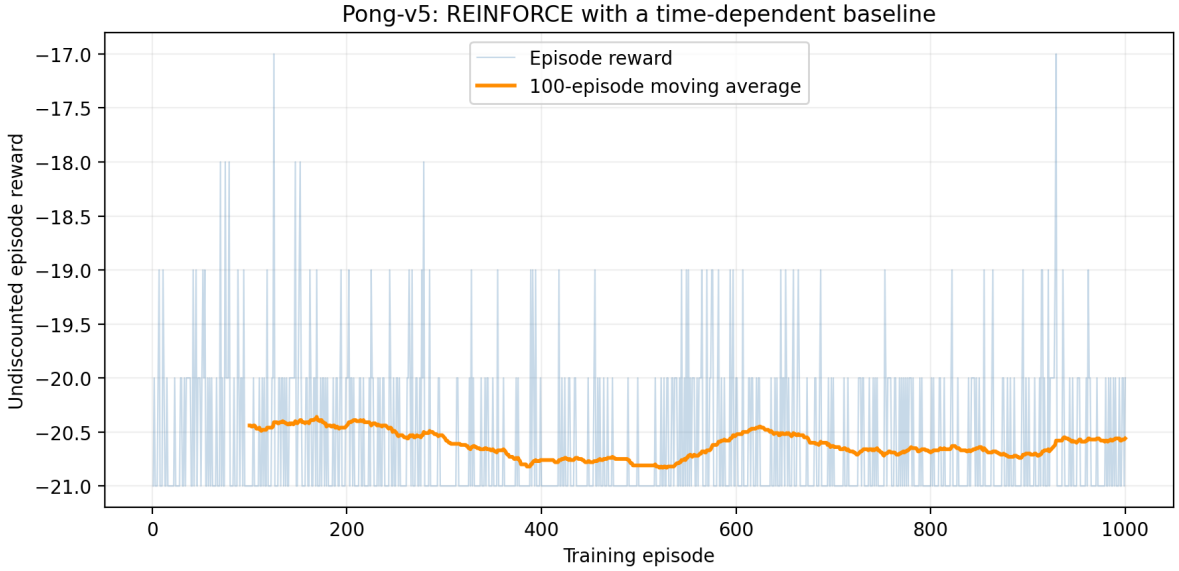


Figure 11. Pong Part 2 training scores with the 100-episode moving average.

First-100 mean score=-20.44; last-100=-20.56; best 100-episode mean=-20.36 (window ending 169). Starting discounted return improved from -1.3700 to -0.8635. Checkpoint 600 was selected on validation mean score (-20.333), not the final test.

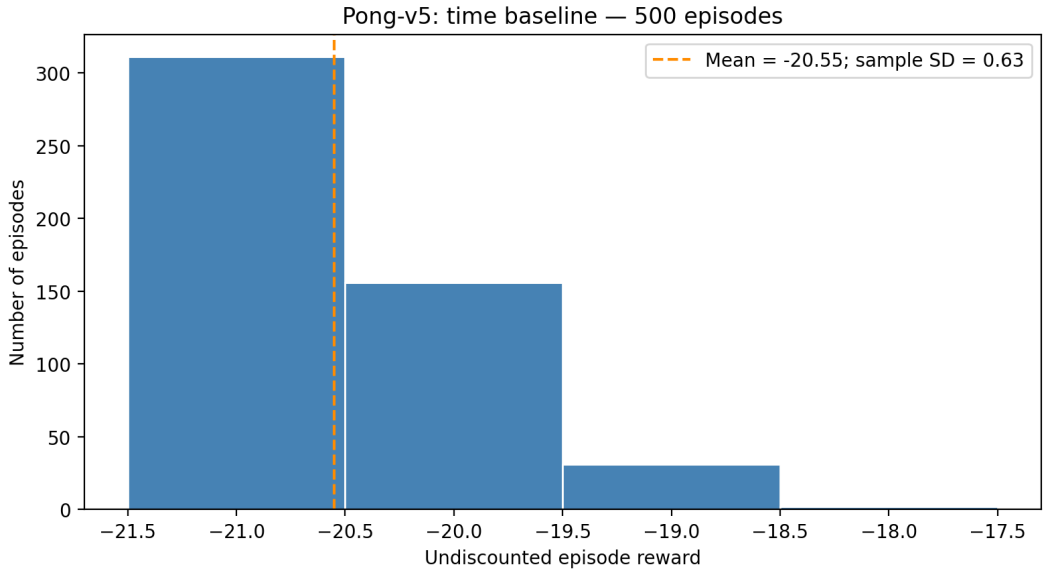


Figure 12. Pong Part 2: 500 held-out scores at checkpoint 600.

Final mean=-20.552; SD=0.629; range -21 to -18; zero truncations; all games lost. The paired score difference (baseline minus no baseline) is -0.146, approximate 95% CI [-0.234,-0.058]. Thus this selected baseline model scored worse, despite the measured gradient-variance reduction. This is not a conclusion that baselines generally harm Pong; this is one initialization in the initial protocol. The separate matched DeepDish experiment follows on the next pages.

9. DeepDish follow-up: Pong without a baseline

After the initial weak Pong results, both methods were rerun from scratch with seed 7, 4,000 training episodes, batch size 8, Adam lr=0.001 and gamma=0.99. This gives 500 optimizer updates per method, versus 250 in the initial runs. No return normalization, point-boundary reset, entropy bonus or altered action set was introduced. Validation used 30 episodes every 400 training episodes.

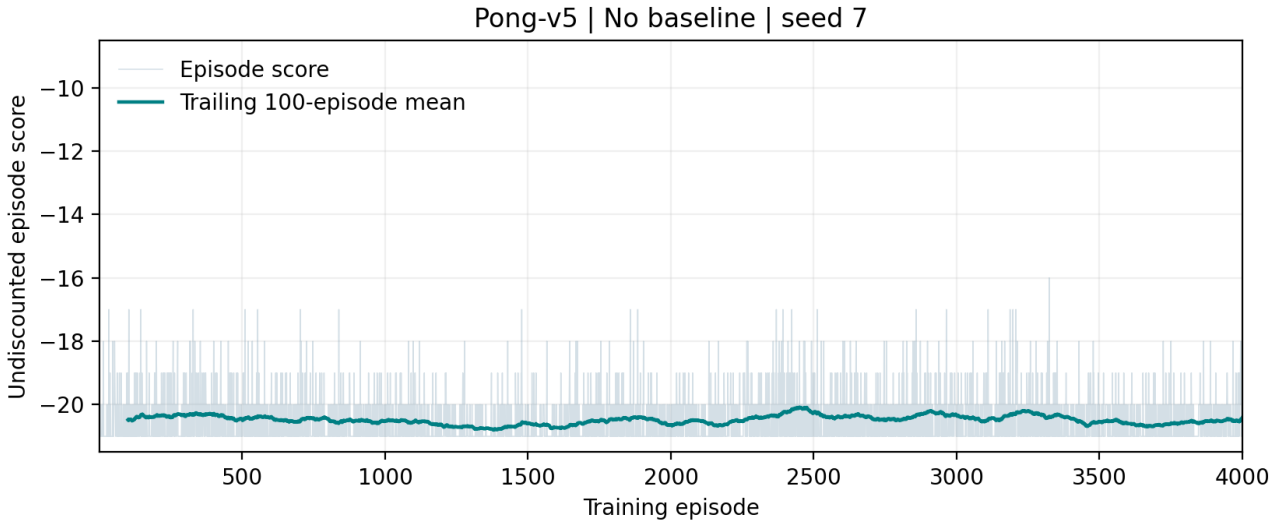


Figure 13. DeepDish Part 1: all 4,000 training scores and the trailing 100-episode mean.

First-100 mean=-20.48; last-100 mean=-20.41; best 100-episode mean=-20.08, ending at episode 2450. The full-game score remains nearly flat. Validation selected episode 4,000 (mean -20.033); this choice was fixed before its 500-episode final test.

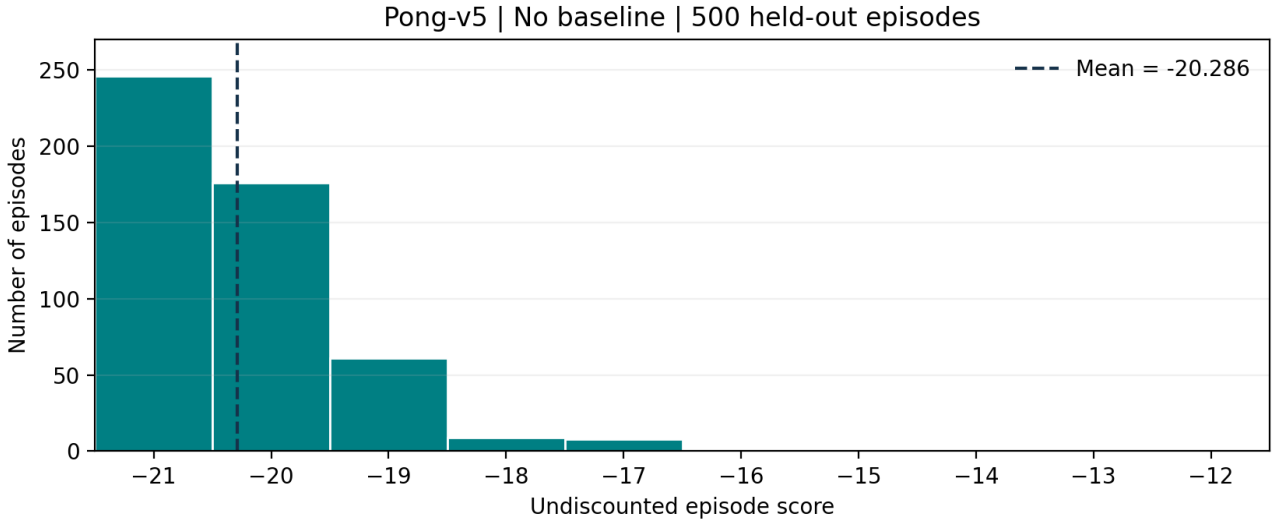


Figure 14. DeepDish Part 1: 500 frozen stochastic-policy scores, selected episode 4,000.

Final mean=-20.286; sample SD=0.865854; approximate 95% CI=[-20.362, -20.210]. Range -21 to -17; 246/500 games score -21; all 500 are losses with no truncation. Mean starting discounted return=-0.938295. This longer-budget configuration also failed to produce effective full-game play.

10. DeepDish follow-up: Pong with a time baseline

This run matches the new no-baseline architecture, initialization seed, training environment seed schedule, optimizer, episode budget, batch size and validation rule. The only algorithmic difference within the pair is the same lagged time EMA baseline (alpha=0.1), computed from previous batches and updated after the actor step.

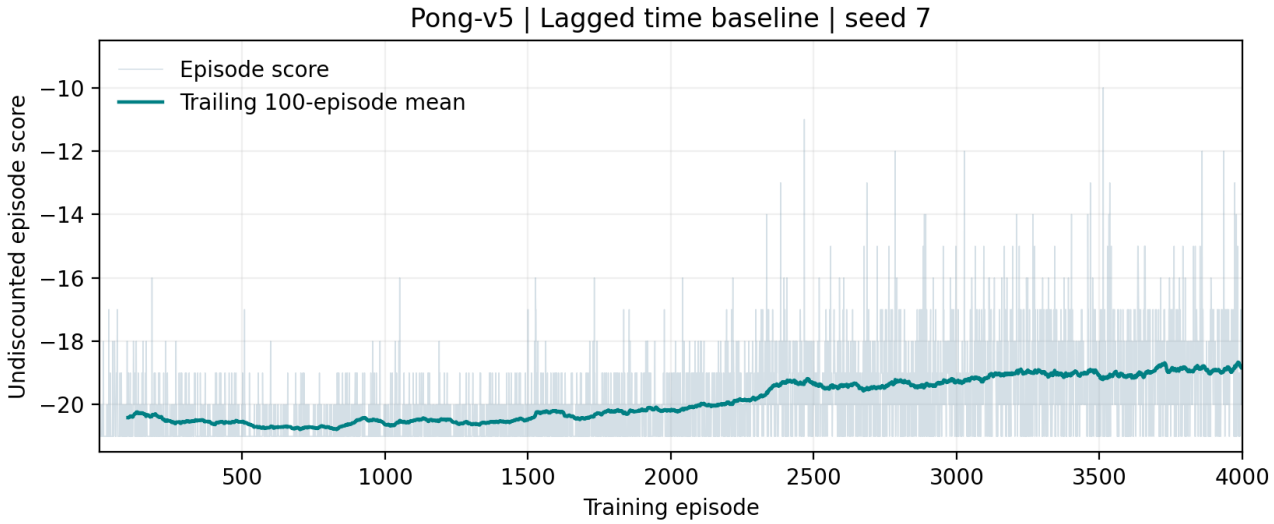


Figure 15. DeepDish Part 2: all 4,000 training scores and the trailing 100-episode mean.

First-100 mean=-20.41; last-100 mean=-18.82; best 100-episode mean=-18.66, ending at episode 3985. Validation selected episode 4,000 (mean -19.100). The late training curve improves, although it is still far from winning.

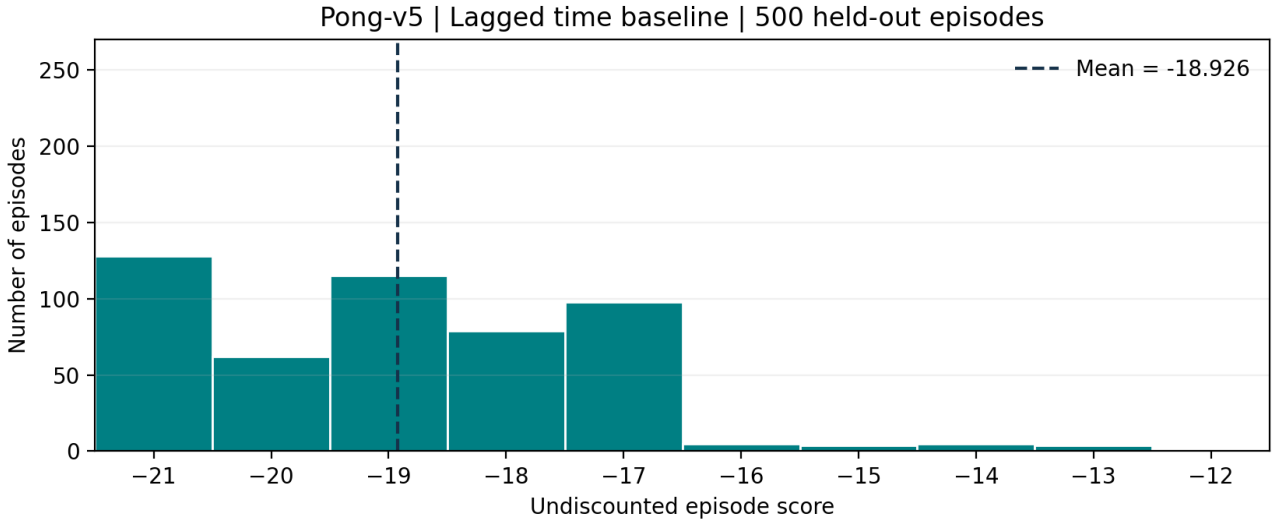


Figure 16. DeepDish Part 2: 500 frozen stochastic-policy scores, selected episode 4,000.

Final mean=-18.926; sample SD=1.687073; approximate 95% CI=[-19.074, -18.778]. Range -21 to -13; 128/500 games score -21; all 500 are losses with no truncation. Mean starting discounted return=0.131972: early discounted rewards can improve substantially while total game scores remain poor.

11. DeepDish comparison, selection and validation

Training episode	No-baseline validation mean	Time-baseline validation mean
400	-20.200	-20.500
800	-20.533	-20.667
1200	-20.733	-20.433
1600	-20.700	-20.300
2000	-20.467	-19.933
2400	-20.133	-19.533
2800	-20.667	-19.667
3200	-20.333	-19.467
3600	-20.567	-19.333
4000	-20.033	-19.100

Within seed 7, the baseline-minus-no-baseline paired mean score difference is 1.360, with approximate 95% CI [1.195, 1.525]. Pairing uses the same 500 environment/action seed identifiers; policies can still generate different trajectories. This supports a positive conditional score difference, not a guarantee across training seeds. Reward SD increases with the baseline; this is not a gradient-variance measurement.

Checks of the changed execution path

Before training, the student ran the adapter correctness check on DeepDish. Pasted console output reports CPU/CUDA relative gradient errors 1.114e-7 (no baseline), 1.100e-7 (baseline), and 1.136e-7 for an 824-step real episode. Matched CPU/hybrid rollouts had identical actions, rewards and processed frames. No optimizer step was taken during these checks. The check can be rerun with --check; it verifies the hardware path, not learning success.

The CPU actor receives current learner weights before every episode, and weights stay fixed through each batch. Only physical GPU 3 was visible as logical cuda:0. Recorded stack: Python 3.11.17, PyTorch 2.7.1+cu126, NumPy 2.1.3, Gymnasium 1.4.0, ALE 0.12.1. Per-run hardware_protocol.json files agree, and all four recorded source hashes match the packaged adapter, runner, loader and notebook.

The final-file audit checks 4,000 ordered training records per run, all ten 30-episode validation sets, seed separation, the selection rule and all 500 raw final scores. It reproduces means, sample SDs and paired intervals; checkpoint ZIP CRCs pass. This packaging runtime has no PyTorch, so fresh tensor loading was not performed here. An optional --load-checkpoints command checks saved model tensors, baseline and all 500 Adam steps on the installed DeepDish environment.

Seed-7 environment seed starts: 20,700,000 training; 30,007,000 validation; 40,007,000 final. Validation/final action seeds start at 307,000/407,000. Both methods share this protocol. Compared with the initial study, seed, batch size, budget and hardware changed together; no single change can be credited for the improvement. No new frozen-gradient variance audit was run.

12. Baseline variance and task-specific edge cases

To isolate the baseline effect from a change in policy, freeze the selected baseline model and its saved baseline. Collect 100 fresh independent trajectories; compute the complete gradient twice on each identical trajectory, once with and once without the baseline. Perform no optimizer updates. The statistic is the trace of sample gradient covariance: the sum of component variances using N-1.

Frozen model	Without baseline	With baseline	Reduction
CartPole, checkpoint 400	18,895.833738	2.545579	99.9865%
Initial Pong, seed 42 / 600	626.465462	88.160239	85.9274%

CartPole statistics were recomputed from the saved 100 x 898 gradient arrays; Pong statistics were recomputed from saved online second moments over 819,586 components. The estimated with/without variance ratios are 0.000134716 and 0.140726. The same trajectories and fixed weights make this a direct estimator comparison, unlike comparing reward SDs of separately trained policies. These local estimates have no confidence intervals on the variance ratio and are not convergence guarantees. These numbers concern the original listed checkpoints, not the new DeepDish models.

CartPole returns saturate near 1/(1-0.95)=20 as episodes lengthen. A time baseline near 20 removes substantial noise but can leave small gradients. Lower measured variance alone does not prove a strong learning signal or justify stopping validation.

Edge case	Handling and observed / deterministic check
Termination vs truncation	Stop on either flag [3]. Treat the 500-step CartPole benchmark as episodic with no bootstrap. Recovery evaluation has 0 and 6 truncations respectively; these are logged, not silently discarded. Pong final tests have 0.
Sparse / signed rewards	Return recurrence handles zeros, losses and wins; no division by episode return or its standard deviation. gamma=0 and gamma=1 cases are checked.
Long / variable episodes	Use only valid trajectory steps; pad the time baseline as needed; leave unvisited indices unchanged. No padded observations enter the loss.
Image mutation / stale history	Preprocess a copy. Verify binary output and signed differences; both consecutive episodes start with a zero difference. A synthetic moving pixel tests motion encoding.
Restricted Pong actions	Map policy indices only to 2 and 3; deterministic fake environments reject extra steps and invalid actions. Real episodes confirm scoring under the restriction.
Memory and gradient chunks	Store Pong differences compactly and recompute log probabilities in chunks of 256 while weights stay fixed. Chunk/full relative gradient error=6.91e-8, below 1e-5.

13. Independent gradient and short-trajectory checks

The strongest gradient check uses a tiny problem with an independently derived expectation, not a second implementation of the same recurrence. Two Bernoulli actions have probabilities p=0.4 and q=0.7, parameterized by logits. Define:

```
r0 = 1 - 2a0;   r1 = 3a0 + 2a1 + 4a0a1
J = 1 - 2p + gamma(3p + 2q + 4pq)
dJ/dtheta0 = p(1-p)[-2 + gamma(3+4q)]
dJ/dtheta1 = gamma q(1-q)(2+4p)
```

Enumerating all four action trajectories with their exact probabilities gives the expected gradient of the actual CartPole loss and Pong chunked loss. Both are compared to -grad J at gamma in {0,0.5,0.95,0.99,1}, with no baseline and fixed b=[1.2,-0.7]: 20 comparisons. Maximum absolute error=4.113e-8, versus tolerance 2e-6. At gamma=0.5, J=2.06 and grad J=[0.216,0.378].

Central finite differences independently check the derivative. A small descent step on the loss increases the exact objective; reversing the step decreases it. Deliberately omitting the outer discount or using an action-dependent baseline produces a gradient error norm greater than 0.05. These negative controls show that the test detects specific mathematical mistakes. A fixed baseline preserves the expected gradient.

Return arithmetic and a first-ten-step observation

Rewards [1,2,3] at gamma=0.5 give [2.75,3.5,3], with zero error. At gamma=0 the returns equal rewards; at gamma=1 they are [6,5,3]. Ten unit rewards at gamma=0.95 give G_0=8.02526122. Below is the saved original untrained 16-step CartPole trace; it is a diagnostic, not a recovered-policy evaluation.

t	Action	P(left)	P(right)	Reward	Return-to-go
0	0	.419	.581	1	11.1975
1	1	.457	.543	1	10.7342
2	0	.420	.580	1	10.2465
3	0	.458	.542	1	9.7332
4	0	.497	.503	1	9.1928
5	0	.534	.466	1	8.6240
6	1	.569	.431	1	8.0253
7	0	.538	.462	1	7.3950
8	1	.572	.428	1	6.7316
9	1	.542	.458	1	6.0333

The recurrence G_t=1+0.95 G_(t+1) and shifting action probabilities are directly visible. Final-step return=1; termination=True; truncation=False. The diagnostic loss was 62.454773 and finite gradient norm 62.428841, with no weight update. Together the suite has 17 passing test methods, zero failures and zero errors. This establishes tested implementation properties, not successful Pong learning.

14. Initial Pong failure investigation: frozen-policy probe

A supplementary review-run experiment evaluated four predeclared frozen policies on 20 fresh episodes each (80 total), without training, model selection or weight updates. Environment seeds were 10,000,000+i and action seeds 100,000+i. The initial 500-episode final evaluations were not altered. This probe tests two explanations: near-deterministic action collapse, and improvement concentrated in early discounted rewards.

Policy	Mean full game score	Mean discounted return	First point won	Mean entropy
Untrained seed 42	-20.55	-1.408737	1/20	0.692492
No baseline, 400	-20.55	-1.232344	1/20	0.610750
Time baseline, 600	-20.45	-0.965021	2/20	0.632002
Time baseline, 1000	-20.45	-0.867826	5/20	0.555252

Interpretation

Simple action collapse is not supported: the first three policies never gave an action probability above 0.99 in the measured states, and the final model did so in about 2.65% of states (mean per-episode fraction). Entropies remain well above zero. Aggregate left/right counts alone would not have tested state-conditional collapse.

The final baseline model won the first point in 5/20 episodes versus 1/20 for the untrained policy, while mean full-game scores stayed near -20.5. This is suggestive of partial early-game learning, not proof of a general success improvement: the sample is small and first-reward timing is not monotonic across checkpoints.

The specified discount gives a reward at t=500 weight 0.99^500=0.00657048. Across the four candidates, mean score after step 500 ranged from -10.65 to -12.30, but its mean contribution to starting discounted return was only about -0.015 to -0.017. Late losses strongly affect reported game score while weakly affecting the optimized start-of-episode objective. This explains why objective improvement need not imply comparable score improvement; it does not identify the sole cause of failure.

What remains unresolved

The initial Pong runs had only 250 optimizer updates. The completed DeepDish follow-up increased this to 500, with a different seed and batch size, and improved baseline scoring without winning games. It does not isolate which cross-protocol change mattered. Controlled batch/budget ablations, frame-stack comparisons and multiple initialization seeds within each protocol remain uncompleted. Original failed learning curves and held-out losses are retained.

This supplementary probe ran in the assistant review runtime, not the original Colab training runtime: Python 3.12.14, NumPy 2.3.5, PyTorch 2.14.1+cpu, Gymnasium 1.4.0 and ALE 0.12.1. Full per-episode diagnostics, first-episode step/scoring traces, source hashes and reproduction commands are in RL_Verified_Evidence.zip. The uploaded Colab test suite also passed independently; the repeated run is not counted as 17 additional distinct tests.

15. Initial checkpoint selection and reproducibility

Selection used maximum validation mean undiscounted reward. Ties used validation discounted return, then the earlier checkpoint for an exact tie. All final evaluations used fresh seeds and frozen stochastic policies, with no training updates or tuning on their results. Validation is a noisy selection tool; it is not a second final test.

Episode	CartPole no b. 50 episodes	CartPole baseline 50 episodes	Pong no b. 30 episodes	Pong baseline 30 episodes
200	42.52	136.26	-20.533	-20.633
400	28.70	344.66	-20.533	-20.733
600	144.52	282.88	-20.700	-20.333
800	98.66	208.34	-20.867	-20.567
1000	104.20	200.88	-20.633	-20.500

CartPole uses initialization seed 42 in both variants. Recovery environment seed starts are 24,200,000 for training, 30,042,000 for validation and 40,042,000 for final evaluation. Validation/final action seeds start at 342,000/442,000 and increment per episode. The random streams are reproducible but trajectories need not remain identical after the policies diverge.

Pong main training starts at environment seed 3,000,000. Final evaluation starts at environment seed 9,000,000 and action seed 90,000, incremented per episode. The paired score comparisons use matching environment/action seed identifiers between methods, not independent training-seed replications.

Recovery provenance and software

The original CartPole checkpoint/CSV backup was unavailable. The recovery reran both methods with the same architecture, learning rate and initialization seed, but changed environment/action seed protocols. Its 141.99/344.44 means must not be mixed with historical 191.86/376.36 notebook means. This is not an exact reconstruction or a new independent initialization-seed study.

Recorded Colab training/recovery stack: Python 3.13.16; NumPy 2.1.3; PyTorch 2.11.0+cpu; Gymnasium 1.4.0; ALE 0.12.1 for Pong; Matplotlib 3.10.0. Versions and source hashes are retained in the result archives. Changes in software or stochastic streams can change numerical trajectories even with a nominal seed.

A metadata issue was found in the original Pong baseline config: its loss-description string omits b_t. Inspection of the executed code confirms that the lagged baseline is passed and subtracted. The correct equation is on page 5. The original archive and its weights were preserved; this is a documented metadata correction, not an experimental change.

Evidence audit

Both original uploaded archives passed integrity checks. Their selected checkpoints load with weights_only=True and finite weights. Every main training log has 1,000 ordered episodes (the separate DeepDish logs each have 4,000). Means and SDs were recomputed from all 500 raw final rewards and matched the saved summaries. Checkpoint choices were rechecked against validation records; gradient-variance statistics were rechecked from arrays/moments.

16. Execution and validation directions

Use the source files supplied with the result archives and the original notebook. Run full training in a fresh directory to avoid overwriting evidence. CPU is sufficient for these recorded runs. In Colab, a shell command below can be run with ! at the start of the cell; execute each command in order. The supplied submission folder already contains recovery/, pong_original/ and review/, so skip extraction there. Use the recorded versions for the relevant experiment; record newer versions separately. DeepDish-specific commands are provided on the next page.

```
python -m pip install -r requirements.txt
python verify_submission.py
python audit_deepdish.py --load-checkpoints
```

A. Fast tests and the independent mathematical oracle

```
python recovery/source/run_checks.py --output checks_rerun
```

This executes 17 test methods without training the games and writes test_log.txt, summary.json, analytical_gradients.csv and software versions. Expected outcome: zero failures/errors; 20 enumerated-gradient comparisons; max error below 2e-6. Inspect the numerical CSV and negative-control tests rather than relying only on a pass flag.

B. Reproduce the primary CartPole recovery experiments

```
python recovery/source/train_additional.py --game cartpole \
  --seeds 42 --episodes 1000 --batch-size 8 --lr 0.01 \
  --checkpoint-every 200 --validation-episodes 50 \
  --output fresh_cartpole/additional_runs
```

Both variants train by default, with matched protocol and independent final holdout. Outputs include training.csv, validation CSVs, selected/latest checkpoints, final_scores.csv, summary and both plots. An interrupted run resumes from the latest completed batch with optimizer/RNG/baseline state; changed configuration requires a new output directory. Do not use a fresh final test to repeatedly tune the same model.

C. Repeat the CartPole frozen-gradient experiment

```
PYTHONPATH=recovery/source python -c "from cp_recovery_tools import \
frozen_gradient_variance; frozen_gradient_variance('recovery',100,42)"
```

Use a working copy of the archive if preserving diagnostic files byte-for-byte. This samples fresh audit trajectories at the selected baseline checkpoint and compares both estimators on identical data; it does not train the actor.

D. Reproduce shuttle, original Pong and the failure probe

Open recovery/source/Assignment1_RL_original.ipynb in a fresh Colab runtime. Run its setup and shuttle section for all three solvers, analytical limit experiments, 5,000-rollout simulation and cap sensitivity. For the original full sequence, run the notebook top to bottom in a clean folder; this also runs the historical CartPole experiments, which are distinct from the recovery results used here. The Pong section contains its pilot, continuation, baseline, checkpoint selection, final evaluation and variance audit.

```
python review/source/pong_diagnostics.py --root pong_original \
  --output probe_rerun --episodes 20
```

The probe reads existing checkpoints, collects 80 diagnostic episodes and writes scores, entropy, reward timing and traces. Expected qualitative check: do not equate improved discounted return with successful full-game play. It is exploratory validation, not another model-selection stage.

17. DeepDish execution and report reproduction

The submitted results are already complete. The audit and report commands do not train, update weights or roll out new test episodes. Run from the repository root. Numerical auditing needs NumPy; plotting additionally needs Matplotlib. The optional tensor audit also needs PyTorch.

```
python audit_deepdish.py --make-plots
python audit_deepdish.py --load-checkpoints
python build_submission_pdf.py \
  --repo https://github.com/Tavishi-Kaushik/rl-assignment-1
```

The audit outputs review/deepdish_audit/summary.json and presentation figures. Expected raw means: -20.286 and -18.926; sample SDs: 0.865854 and 1.687073; both selected episodes: 4,000. With --load-checkpoints, model shapes and finite weights, saved configs/baselines, equal selected/latest weights, and 500 Adam steps are checked. ZIP integrity alone does not establish tensor correctness.

Reproduce the single-GPU path and a fresh experiment

Use the installed DeepDish environment and an allocated single compatible GPU. The recorded run used device 3, a GTX 1080, and PyTorch 2.7.1+cu126. Visibility is not GPU reservation; follow the shared-server allocation policy. Do not run training over submitted evidence. The adapter requires a passing --check with matching software/hardware protocol before allowing --train.

```
export CUDA_DEVICE_ORDER=PCI_BUS_ID
export CUDA_VISIBLE_DEVICES=3
export OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1
python deepdish_pong.py --check
python deepdish_pong.py --train --seeds 7 \
  --variants no_baseline time_baseline \
  --episodes 4000 --batch-size 8 --lr 0.001 \
  --checkpoint-every 400 --validation-episodes 30 \
  --output fresh_deepdish/paired_seed7_b8_e4000
```

Each variant performs 4,000 training episodes, selects on validation, then runs 500 held-out episodes. Resume uses the same output path and exact arguments; changed source hashes, software or budgets are rejected. CPU Atari simulation and action sampling can dominate runtime, so low instantaneous GPU utilization is not by itself evidence of a stalled learner.

Submission checklist

Review this report and the source; push the updated source, raw results and figures to Tavishi-Kaushik/rl-assignment-1; ensure the grader has access to this private repository; submit the revised PDF on Canvas. The scripts do not change repository visibility, invite graders, publish commits or submit to Canvas.

18. Additional three-month validation plan

Month 1: replication and independent correctness checks

Train at least ten independent initialization seeds per configuration under a predeclared environment-step budget, with disjoint validation/test seeds. Report per-seed means, medians, failure rates and bootstrap intervals over training seeds. Keep sampled and greedy policy evaluations separate. Extend the exact-gradient oracle to state-dependent transitions, stochastic rewards and variable horizons. For the shuttle, compare against an independent linear-programming solver and exhaustive tiny-horizon problems with nonuniform arrival probabilities.

Month 2: controlled tuning and baseline comparisons

Change one factor at a time: learning rate, batch size, training budget, network width and observation representation. Compare the lagged time baseline with a state-value baseline trained separately, with detached actor targets. Measure gradient covariance, signal-to-noise and true objective improvement across multiple checkpoints. Compare equal episode budgets and equal optimizer-update budgets separately: larger batches otherwise confound gradient noise with fewer updates.

Month 3: robustness and the reason Pong fails

Compare frame stacks with frame differences while retaining the prescribed actions and discounted objective. Record first-point success, total score, reward timing and late-episode gradients. Use deterministic or reduced-stochasticity variants only as labeled diagnostics, then test the original v5 environment with sticky actions. Point-reset returns or omission of the outer discount would change the estimator/objective and must be separate ablations, not silent fixes. Evaluate the prespecified chosen protocol once on fresh held-out seeds.

For the shuttle, vary starting queues near the cap, cap size, bursty arrivals, dispatch/holding costs and explicit overflow penalties. Check whether the threshold structure survives, whether value bounds remain valid, and whether low-state insensitivity persists under increased arrival load. These are proposed tests, not completed results.

Conclusion

The shuttle results have strong analytical and numerical agreement. The policy-gradient implementations pass independent exact-gradient and task-specific checks. The baseline substantially reduces empirical gradient variance and the recovered CartPole baseline model has a higher evaluated mean reward. Neither CartPole model reliably reaches 500. The matched DeepDish follow-up yields a 1.36-point baseline advantage in Pong, but still no wins in either 500-game test. More independent training seeds and controlled tuning are needed before claiming robust learning performance.

References, evidence and assistance disclosure

The assignment handout supplies the shuttle parameters, required environments, discount factors, plots, final-evaluation budget and validation/report requirements. All numerical experiment results are from the supplied logs/notebook or the explicitly identified review-run diagnostic; web documentation is not a source for reported scores.

[1] Farama Foundation. Gymnasium: Cart Pole environment documentation.

https://gymnasium.farama.org/environments/classic_control/cart_pole/

[2] Farama Foundation. Arcade Learning Environment: Pong documentation.

https://ale.farama.org/environments/pong/

[3] Farama Foundation. Gymnasium Env API: step, reset and episode endings.

https://gymnasium.farama.org/api/env/

[4] PyTorch. torch.distributions: categorical sampling and log probabilities.

https://docs.pytorch.org/docs/2.14/distributions.html

Documentation accessed October 26, 2026. Current API references do not replace the recorded training-version information.

Evidence files

Artifact	Contents and use
cartpole_recovery_results.zip	New CartPole source, training/validation/final CSVs, selected models, figures, 100-trajectory gradient arrays, executed validation logs and historical notebook evidence.
pong_results_backup.zip	Original Pong pilot/main logs, baseline/no-baseline checkpoints, final 500-episode scores, figures and variance moments.
RL_Verified_Evidence.zip	Recomputed audit, local 17-test rerun, uploaded Colab checks, all 80 supplementary Pong probe episodes, source and provenance hashes.
DeepDish_Pong_Final_Results.zip	Both completed seed-7 runs: 4,000 training records each, ten validation sets each, 500 final scores each, selected/latest checkpoints, figures and exact hardware/source protocols. Included under deepdish_runs/.

The original notebook is preserved in the CartPole archive source/ directory. The GitHub repository should include that notebook and all scripts used for training, validation and analysis, plus instructions and the small numerical/figure outputs. Large checkpoint backups can be provided separately, with a clear retrieval location. Repository access must be available to the grader.

Assistance disclosure

ChatGPT assisted with implementation, debugging, test design, interpretation and report preparation. The main training and final evaluation results were produced in the student Colab workflow and retained in the supplied archives. The matched 4,000-episode follow-up was executed by the student on DeepDish using one GPU. The additional 80-episode frozen-policy Pong probe and independent test rerun were executed in the assistant review runtime and are labeled accordingly. Unrun multi-seed and controlled-ablation experiments are presented only as future work. This assistance must be disclosed in accordance with the course policy.